

圖層與基本繪圖

李東霖 博士

國立臺灣海洋大學 電機工程學系

2025 Fall semester

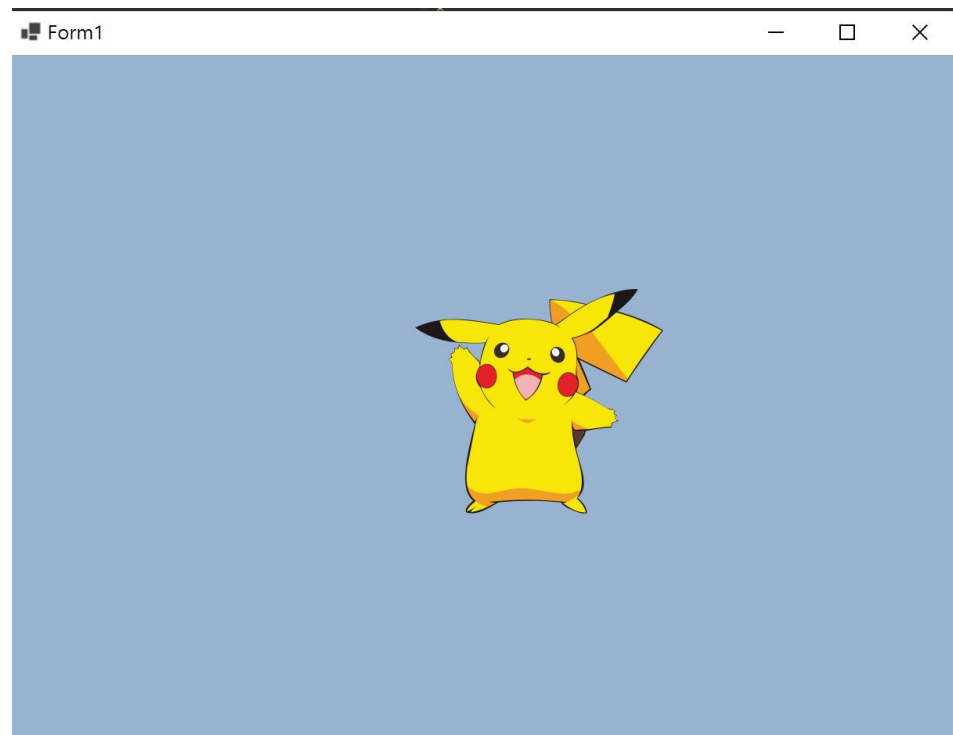
Panel元件常用屬性與應用

- 和GroupBox 非常類似，但Panel沒有 Text 屬性。
- 另外Panel有捲軸屬性，而 GroupBox 沒有。
- 常用屬性
 - AutoScroll:用來設定當物件大小超過面板大小時，是否自動顯示捲軸。
 - BorderStyle:用來設定面板邊框的樣式。若屬性值為 **None** 表示此面板無邊框、**FixedSingle** 設成固定單線框、**Fixed3D** 設成固定立體框。
 - AutoScrollMargin:當 AutoScroll = True 時，透過此屬性可用來調整面板四周邊界大小。
 - AutoScrollMinSize:自動捲動區域的最小大小。
- 常用事件
 - Paint():繪圖使用

Panel應用(1) – 透明背景的元素

- 需要一張有透明背景的PNG檔
- 設定panel的背景顏色為透明
`panel1.BackColor = Color.Transparent;`
- 設定panel的父元件為視窗表單
`panel1.Parent = this;`

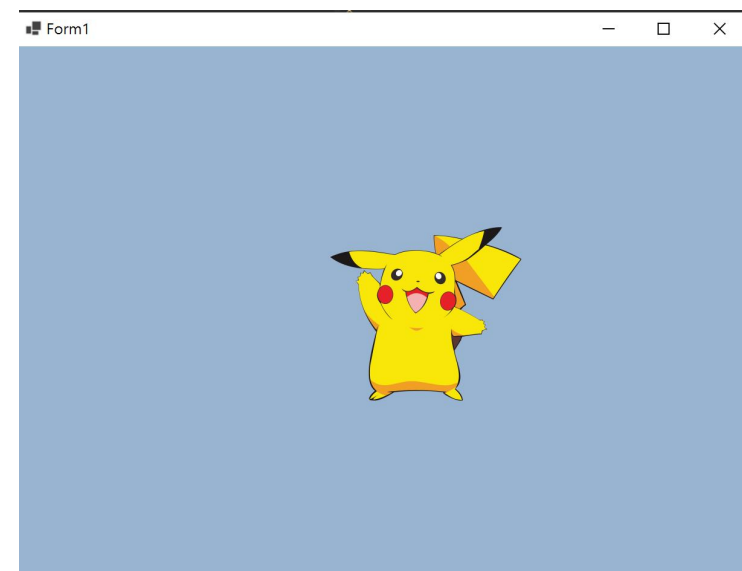
註:其它元件同理



Panel應用(2) – 用滑鼠在視窗中移動Panel

```
private Point mouse_offset;
private void panell1_MouseDown(object sender, MouseEventArgs e)
{
    mouse_offset = new Point(-e.X, -e.Y); //紀錄點擊元件的位置
}

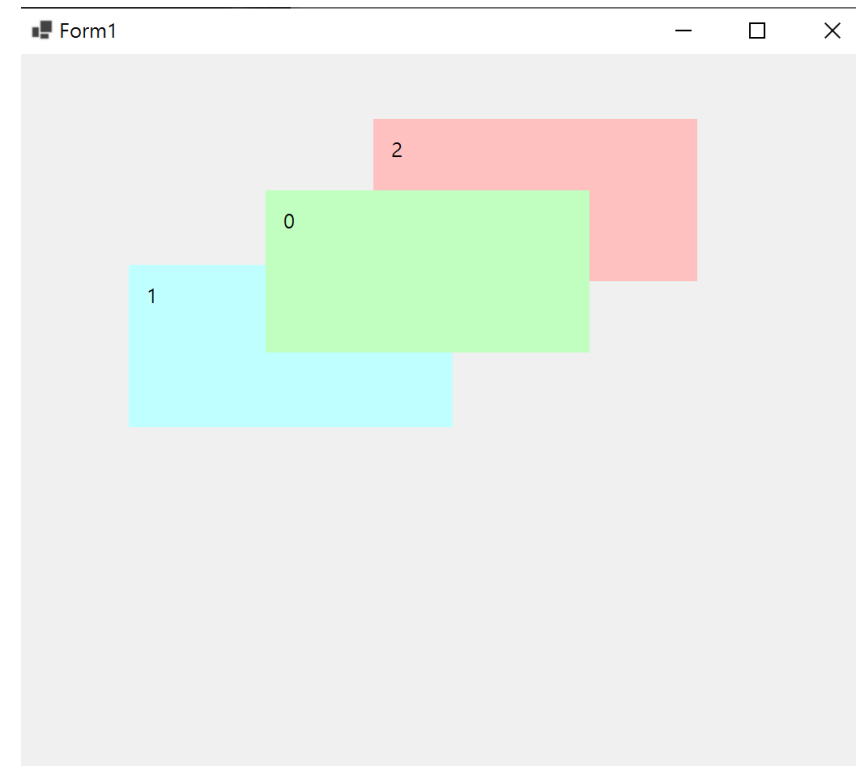
private void panell1_MouseMove(object sender, MouseEventArgs e)
{
    if (e.Button == MouseButtons.Left)
    {
        Panel p = (Panel)sender;
        //Control.MousePosition可取得滑鼠在螢幕中的座標值
        //但因為元件是在Form視窗，所以須透過PointToClient方法把螢幕做標值轉換為Form視窗中的座標值。
        Point mousePos = PointToClient(Control.MousePosition);
        //修正元件在移動中與滑鼠的位置，避免元件總會落在滑鼠的右下方。
        mousePos.Offset(mouse_offset.X, mouse_offset.Y);
        //限制元件移動別超出視窗
        mousePos.X = Math.Min(Math.Max(mousePos.X, 0), this.Size.Width - panell1.Size.Width);
        mousePos.Y = Math.Min(Math.Max(mousePos.Y, 0), this.Size.Height - panell1.Size.Height);
        p.Location = mousePos;
    }
}
```



註:其它元件同理

Panel應用(3) – 圖層切換

```
private void Form1_Load(object sender, EventArgs e)
{
    label1.Text = this.Controls.GetChildIndex(panel1).ToString();
    label2.Text = this.Controls.GetChildIndex(panel2).ToString();
    label3.Text = this.Controls.GetChildIndex(panel3).ToString();
}
private void panel1_MouseClick(object sender, MouseEventArgs e)
{
    if (e.Button == MouseButtons.Left)
        this.Controls.SetChildIndex(panel1, this.Controls.GetChildIndex(panel1) - 1);
    else
        this.Controls.SetChildIndex(panel1, this.Controls.GetChildIndex(panel1) + 1);
    label1.Text = this.Controls.GetChildIndex(panel1).ToString();
}
private void panel2_MouseClick(object sender, MouseEventArgs e)
{
    if (e.Button == MouseButtons.Left)
        this.Controls.SetChildIndex(panel2, this.Controls.GetChildIndex(panel2) - 1);
    else
        this.Controls.SetChildIndex(panel2, this.Controls.GetChildIndex(panel2) + 1);
    label2.Text = this.Controls.GetChildIndex(panel2).ToString();
}
private void panel3_MouseClick(object sender, MouseEventArgs e)
{
    if (e.Button == MouseButtons.Left)
        this.Controls.SetChildIndex(panel3, this.Controls.GetChildIndex(panel3) - 1);
    else
        this.Controls.SetChildIndex(panel3, this.Controls.GetChildIndex(panel3) + 1);
    label3.Text = this.Controls.GetChildIndex(panel3).ToString();
}
```



Tip:
`Control.BringToFront();`//移到最上層
`Control.SendToBack();`//移到最下層

Graphics畫布

- 當在表單或控制項建立Graphics物件後，就可以在表單或控制項中繪圖
 - 包括：繪製文字、直線、矩形等，也可以將圖形填滿。
- 繪圖座標系統的基本單位是像素（**Pixels**），畫布的繪圖區域為一個矩形，以左上角為原點(0.0)，X軸作為由左至右，Y軸座標由上至下。
- 建立Graphics方式
 1. 使用 CreateGraphics 方法建立 Graphics 物件
`Graphics g = this.CreateGraphics();`
 2. 從 Image 物件建立
`Graphics g = Graphics.FromImage('檔案名稱');`
- 清除畫布
 - `Graphics.Clear(Color.Gray);`//清除後用Color.Gray填滿
- 記憶體釋放
 - `Graphics.Dispose();`

Graphics的基本繪圖(1) - 建立畫筆

- 畫筆顏色與大小

```
Color color= Color.FromArgb(0,0,0,0); //透明質(透明0-255不透明),R,G,B  
System.Drawing.Pen myPen;  
myPen = new System.Drawing.Pen(color, 5); //筆粗為5
```

- 畫筆樣式

```
myPen.DashStyle = DashStyle.Dot;    //設定點線  
myPen.DashStyle = DashStyle.Dash;    //設定虛線  
myPen.DashStyle = DashStyle.DashDot; //設定點虛線  
myPen.StartCap = LineCap.ArrowAnchor; //起點Cap設為 ArrowAnchor 樣式  
myPen.StartCap = LineCap.Flat;        //起點Cap設為 Flat 樣式  
myPen.EndCap = LineCap.DiamondAnchor; //起點Cap設為 DiamondAnchor 樣式  
myPen.EndCap = LineCap.Triangle;      //起點Cap設為 ArrowAnchorTriangle 樣式
```

- 實心筆刷顏色(用於填滿圖形)

- SolidBrush myBrush = new SolidBrush(Color.Black);

Graphics的基本繪圖(2) – 基本圖形繪製

- 繪製直線
`Graphics.DrawLine(myPen, 20, 10, 300, 100);`
- 繪製矩形
`Graphics.DrawRectangle(myPen, 0, 0, 100, 100);` //以左上座標(x,y)+高度+寬度畫一空心矩型。
`Graphics.FillRectangle(myBrush, new Rectangle(0, 0, 200, 300));` //畫一實心矩型
- 繪製橢圓形
`Graphics.DrawEllipse(myPen, 0, 0, 100, 100);`
`Graphics.FillEllipse(myBrush, new Rectangle(0, 0, 200, 300));`
- 繪製文字
`System.Drawing.Font myFont = new System.Drawing.Font("Arial", 16);`
`float x = 150.0F;`
`float y = 50.0F;`
`System.Drawing.StringFormat drawFormat = new System.Drawing.StringFormat();`
`Graphics.DrawString("Sample Text", myFont, myBrush, x, y, drawFormat);`
- 繪製垂直文字
`drawFormat.FormatFlags = StringFormatFlags.DirectionVertical;`
`Graphics.DrawString("Sample Text", myFont, myBrush, x, y, drawFormat);`

Graphics的基本繪圖(3) – 自訂形狀繪製

- 繪製空心形狀

```
Point[] points = {  
    new Point(0, 100),  
    new Point(50, 80),  
    new Point(100, 20),  
    new Point(150, 80),  
    new Point(200, 100)};  
Graphics.DrawPolygon(myPen, points);
```

- 繪製實心形狀

```
Graphics.FillPolygon(myBrush, points);
```

Graphics的基本繪圖(4) – 曲線繪製

- 繪製開放曲線

```
Point[] points = {  
    new Point(0, 100),  
    new Point(50, 80),  
    new Point(100, 20),  
    new Point(150, 80),  
    new Point(200, 100)};  
Graphics.DrawCurve(myPen, points);
```

- 繪製封閉曲線

```
Graphics.DrawClosedCurve(myBrush, points);
```

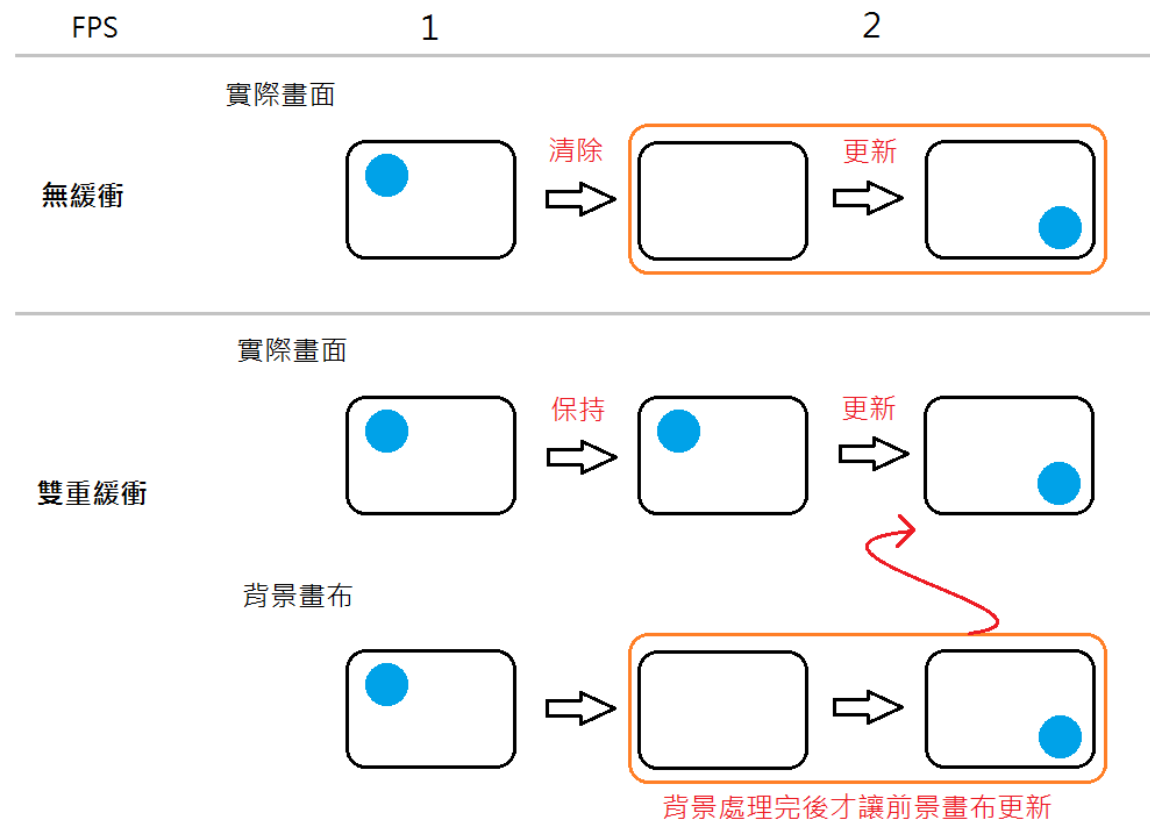
Graphics的基本繪圖(5) – 顯示繪圖

- 在Panel或PictureBox元件上顯示(使用Paint()事件)

```
private void panell1_Paint(object sender, PaintEventArgs e)
{
    Graphics g = e.Graphics;
    g.FillRectangle(Brushes.Blue, 50, 50, 100, 100);
}
```

雙緩衝區畫布

- 當畫面所需繪圖的物件過多時或是欲繪製的圖形較大，畫面可能會有閃爍的情形發生，為了避免此問題可以用 **Buffered Graphics** 來繪製圖形
- 其概念為準備兩個繪圖物件，其中一個是前景區（看到的畫面），另一個繪圖物件（看不到的）。在畫面準備好之前，所有的圖形先繪製在您看不到的畫面，當所有的圖形都繪製完畢，再將所有的圖形繪製到前景，此時前景會整個被後繪圖區所覆蓋，完成一次畫面的播放



Graphics的基本繪圖(6) – 雙緩衝繪圖

- 將表單的DoubleBuffered 屬性設定為 true
- 加入一個timer元件並在Tick()事件中加入以下程式碼

```
BufferedGraphicsContext currentContext = BufferedGraphicsManager.Current;  
BufferedGraphics myBuffer = currentContext.Allocate(this.CreateGraphics(), this.DisplayRectangle);
```

```
//清除繪圖畫面，並用原本的背景色填充，否則背景呈現黑色  
myBuffer.Graphics.Clear(this.BackColor);
```

```
//在圖形緩衝區中畫圖  
myBuffer.Graphics.DrawEllipse(Pens.Blue, x, y, 70, 70);
```

```
//將圖形緩衝區的內容畫到指定的畫布上  
myBuffer.Render(this.CreateGraphics());
```

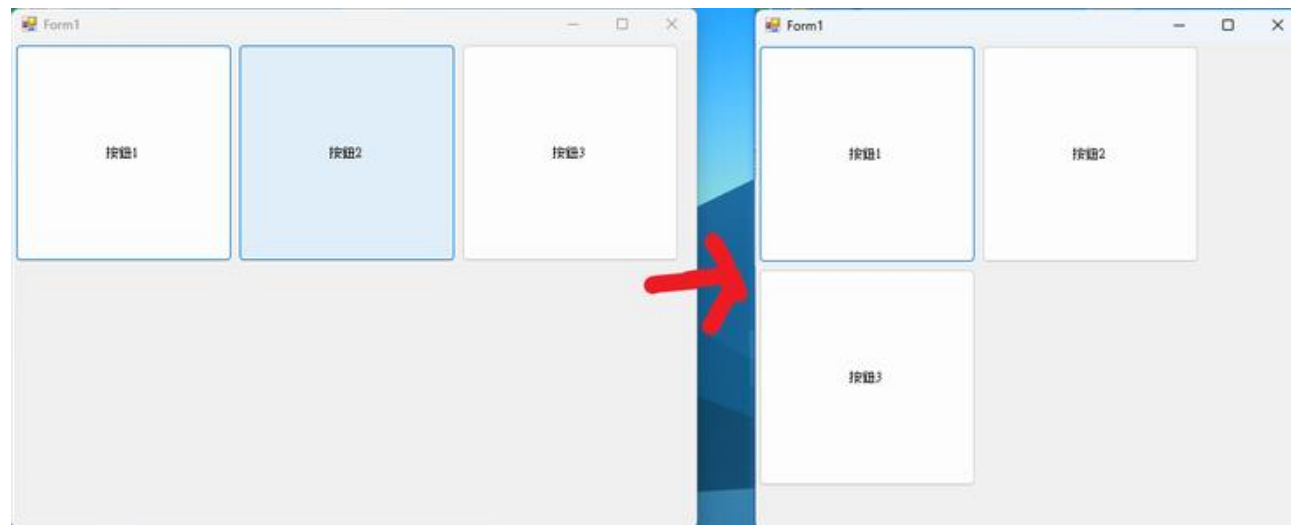
```
//釋放緩衝區的資源  
myBuffer.Dispose();  
this.Refresh(); // 更新視窗  
this.Invalidate(); //更新元件內容
```

TableLayoutPanel 元件常用屬性與應用

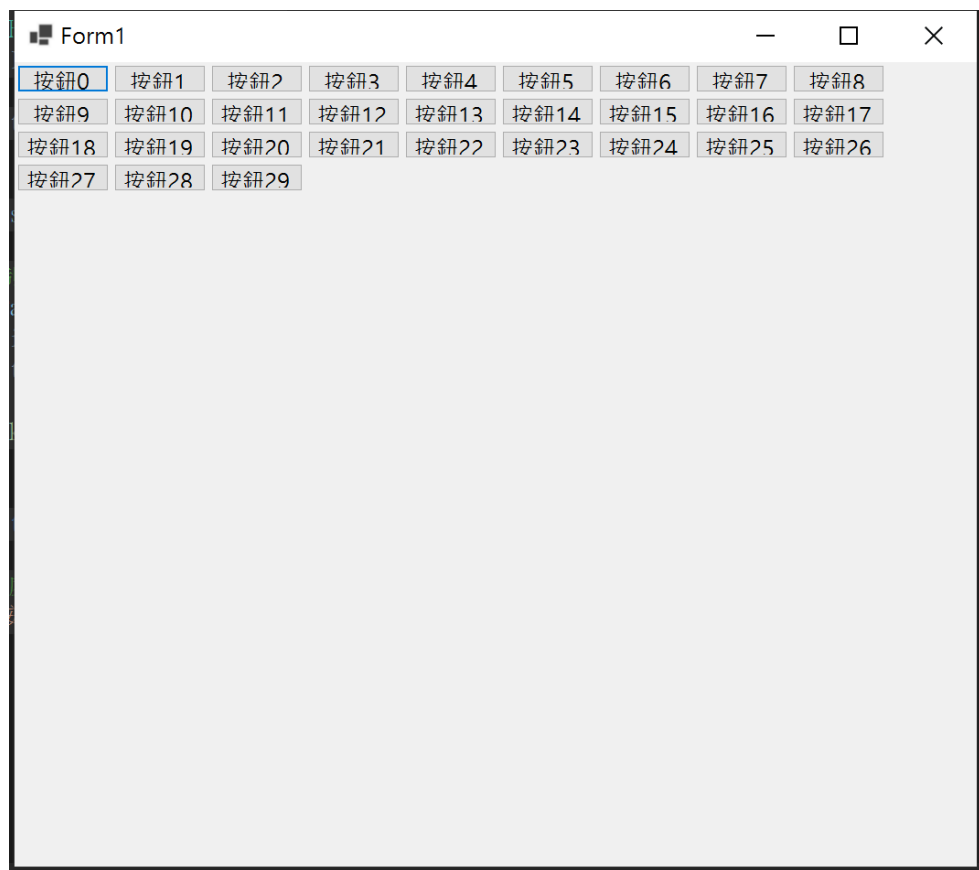
- 常用屬性

- 配置

- ColumnCount：直行數量
 - Columns：設定行大小
 - RowCount：橫列數量
 - Rows：設定列大小
 - Size：元件預設大小
 - FlowDirection 屬性指定子控制項排列的方向。
 - WrapContents 屬性指定是否在達到容器邊界時換行。
 - AutoScroll 屬性指定當容器不足以顯示所有子控制項時，是否自動顯示捲軸。



TableLayoutPanel 範例



```
public Form1()
{
    InitializeComponent();
    Button[] b = new Button[30];
    for (int i = 0; i < b.Length; ++i)
    {
        b[i] = new Button();
        b[i].Text = "按鈕" + i.ToString();
        b[i].Size = new System.Drawing.Size(100, 30);
        b[i].Click += new EventHandler(button_Click);
        flowLayoutPanel1.Controls.Add(b[i]);
    }
    this.Controls.Add(flowLayoutPanel1);
}

private void Form1_Load(object sender, EventArgs e)
{
    // 建立 FlowLayoutPanel 控制項
    FlowLayoutPanel flowLayoutPanel1 = new FlowLayoutPanel();
    flowLayoutPanel1.FlowDirection = FlowDirection.LeftToRight;
    flowLayoutPanel1.WrapContents = true;
    flowLayoutPanel1.AutoScroll = true;
    flowLayoutPanel1.Dock = DockStyle.Fill; // 佔滿整個視窗
}

private void button_Click(object sender, EventArgs e)
{
    // 在這裡可以寫點擊按鈕後的處理邏輯
    MessageBox.Show($"你按下了按鈕：{((Button)sender).Text}");
}
```